

Task 07 Web Application Vulnerability Testing

Chapter 1. Introduction to Web Application Security

Web applications play a critical role in modern digital systems and are widely used for services such as online banking, e-commerce, education, healthcare, and enterprise operations. Due to their accessibility over the internet, web applications are frequent targets for cyberattacks. Web application security focuses on protecting these applications from vulnerabilities that can be exploited to gain unauthorized access, manipulate data, or disrupt services.

Web applications typically consist of a client side (browser), a server side (web server and application logic), and a backend database. Communication between these components occurs through HTTP and HTTPS requests. If user inputs are not properly validated or securely handled, attackers can exploit weaknesses in the application logic, leading to serious security breaches.

Common web application attacks include SQL Injection, Cross-Site Scripting (XSS), authentication bypass, insecure session management, and misconfiguration issues. These vulnerabilities can allow attackers to access sensitive information, modify database records, hijack user sessions, or execute malicious scripts in a victim's browser.

With the increasing complexity of web applications, understanding web application security has become essential for developers and security professionals. Identifying vulnerabilities early and testing applications in a controlled environment helps organizations improve security posture and reduce the risk of real-world attacks. Tools such as vulnerable test applications and security testing platforms are widely used to study and analyze these risks in a practical manner.

This task focuses on understanding web application security concepts through hands-on vulnerability testing using a deliberately vulnerable application. By analyzing real attack scenarios, common weaknesses in web applications can be better understood along with their potential impact and the importance of secure development practices.

Chapter 2. Overview of OWASP Top 10

The **OWASP Top 10** is a globally recognized framework that identifies the most critical security risks affecting web applications. It is developed and maintained by the Open Web Application Security Project (OWASP) and serves as a standard reference for developers, security professionals, and organizations to understand and mitigate common web application vulnerabilities.

The OWASP Top 10 list highlights vulnerabilities based on their prevalence, severity, and potential impact. These risks often arise due to insecure coding practices, improper input validation, weak authentication mechanisms, and misconfigured application components. The framework is periodically updated to reflect emerging threats and changes in modern web technologies.

Some of the commonly identified risks in the OWASP Top 10 include injection flaws, broken authentication, sensitive data exposure, security misconfiguration, and cross-site scripting. These vulnerabilities can allow attackers to manipulate application behavior, access confidential data, or compromise user sessions.

2.1 OWASP Top 10 Vulnerabilities (2025)

According to the OWASP Top 10 (2025), the most critical web application security risks are:

1. **A01:2025 – Broken Access Control**
Occurs when users can access resources or perform actions beyond their intended permissions.
2. **A02:2025 – Security Misconfiguration**
Results from improper security settings, default configurations, or incomplete system hardening.
3. **A03:2025 – Software Supply Chain Failures**
Vulnerabilities introduced through third-party libraries, dependencies, or compromised software components.
4. **A04:2025 – Cryptographic Failures**
Involves improper use or absence of encryption, leading to exposure of sensitive data.
5. **A05:2025 – Injection**
Happens when untrusted input is interpreted as commands or queries, such as SQL Injection.
6. **A06:2025 – Insecure Design**
Arises from fundamental design flaws where security controls are missing or insufficient.
7. **A07:2025 – Authentication Failures**
Weak authentication mechanisms that allow attackers to compromise user accounts.

8. **A08:2025 – Software or Data Integrity Failures**

Occurs when applications rely on untrusted updates, plugins, or data sources.

9. **A09:2025 – Security Logging and Alerting Failures**

Insufficient logging and monitoring that prevents timely detection of attacks.

10. **A10:2025 – Mishandling of Exceptional Conditions**

Improper error handling that reveals system details or leads to unexpected behavior.

In this task, emphasis is placed on two critical OWASP Top 10 vulnerabilities: **SQL Injection** and **Cross-Site Scripting (XSS)**. These vulnerabilities are among the most frequently exploited issues in real-world web applications and demonstrate how improper handling of user input can lead to severe security consequences.

Understanding the OWASP Top 10 provides a structured approach to identifying, categorizing, and mitigating web application vulnerabilities. It also helps in aligning security testing activities with industry-recognized standards, making vulnerability assessment more systematic and effective.

Chapter 3. Lab Setup – DVWA and Burp Suite

To perform practical web application vulnerability testing, a controlled and intentionally vulnerable lab environment was set up. This environment allows safe experimentation with common web vulnerabilities without risking real-world systems.

3.1 Kali Linux Environment

The testing environment was built using **Kali Linux**, a security-focused operating system widely used for penetration testing and ethical hacking. Kali Linux provides pre-installed tools required for web application testing, including Burp Suite, database services, and web servers.

Apache web server and MariaDB database services were configured and started to support the deployment of the vulnerable application.

Figure 3.1: Kali Linux Testing Environment

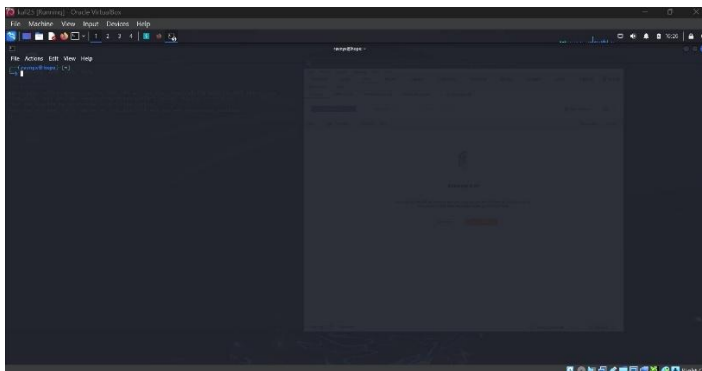


Figure 3.1.1: Apache and MariaDB Services Running

```
File Machine View Input Devices Help
Trach
ravnyx@hope: ~
File Actions Edit View Help
(ravnyx@hope)~$ sudo mv /etc/mysql/mariadb.conf.d/provider_bzip2.cnf /etc/mysql/mariadb.conf.d/provider_bzip2.cnf.disabled
(ravnyx@hope)~$ sudo mv /etc/mysql/mariadb.conf.d/provider_lz4.cnf /etc/mysql/mariadb.conf.d/provider_lz4.cnf.disabled
(ravnyx@hope)~$ sudo systemctl restart mariadb
(ravnyx@hope)~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.4.2 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-01-26 13:58:29 IST; 14s ago
  Invocation: c829f1124e09a259e1050c38567a9d
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 33590 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysql (code=exited, status=0/
   Process: 33592 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/
   Process: 33594 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR="" || VAR="/usr/bin/galera_recovery" ; /usr
   Process: 33647 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=
   Process: 33649 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 33625 (mariadb)
    Status: "Taking your SQL requests now..."
       Tasks: 14 (limit: 68476)
      Memory: 220.1M (peak: 224.7M)
         CPU: 1.787s
    CGroup: /system.slice/mariadb.service
            └─33625 /usr/sbin/mariadb

Jan 26 13:58:28 hope mariadb[33625]: 2026-01-26 13:58:28 0 [Note] Plugin 'FEEDBACK' is disabled.
Jan 26 13:58:28 hope mariadb[33625]: 2026-01-26 13:58:28 0 [Note] Plugin 'wsrep-provider' is disabled.
Jan 26 13:58:28 hope mariadb[33625]: 2026-01-26 13:58:28 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql
Jan 26 13:58:28 hope mariadb[33625]: 2026-01-26 13:58:28 0 [Note] InnoDB: Buffer pool(s) load completed at 260126
Jan 26 13:58:29 hope mariadb[33625]: 2026-01-26 13:58:29 0 [Note] Server socket created on IP: '127.0.0.1'.
Jan 26 13:58:29 hope mariadb[33625]: 2026-01-26 13:58:29 0 [Note] mariadb: Event Scheduler: Loaded 0 events
Jan 26 13:58:29 hope mariadb[33625]: 2026-01-26 13:58:29 0 [Note] /usr/sbin/mariadb: ready for connections.
Jan 26 13:58:29 hope mariadb[33625]: Version: '11.4.2-MariaDB-4' socket: '/run/mysql/mysql.sock' port: 3306
Jan 26 13:58:29 hope systemd[1]: Started mariadb.service - MariaDB 11.4.2 database server.
Jan 26 13:58:29 hope /etc/mysql/debian-start[33668]: Triggering mysam-recover for all MyISAM tables and aria-reco
lines 1-29/29 (END)

ravnyx@hope: ~
File Actions Edit View Help
(ravnyx@hope)~$ sudo systemctl start apache2
(ravnyx@hope)~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-01-26 13:42:57 IST; 3min 8s ago
  Invocation: f9e3aa0bdbaf41fd81fe34bda73fd3ba
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 25339 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 25362 (apache2)
    Tasks: 6 (limit: 10375)
      Memory: 19.5M (peak: 19.7M)
         CPU: 199ms
    CGroup: /system.slice/apache2.service
            └─25363 /usr/sbin/apache2 -k start
              25366 /usr/sbin/apache2 -k start
              25367 /usr/sbin/apache2 -k start
              25368 /usr/sbin/apache2 -k start
              25369 /usr/sbin/apache2 -k start
              25370 /usr/sbin/apache2 -k start

Jan 26 13:42:57 hope systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 26 13:42:57 hope apachectl[25362]: AH00558: apache2: Could not reliably determine the server's fully qualified
Jan 26 13:42:57 hope systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

3.2 DVWA (Damn Vulnerable Web Application) Setup

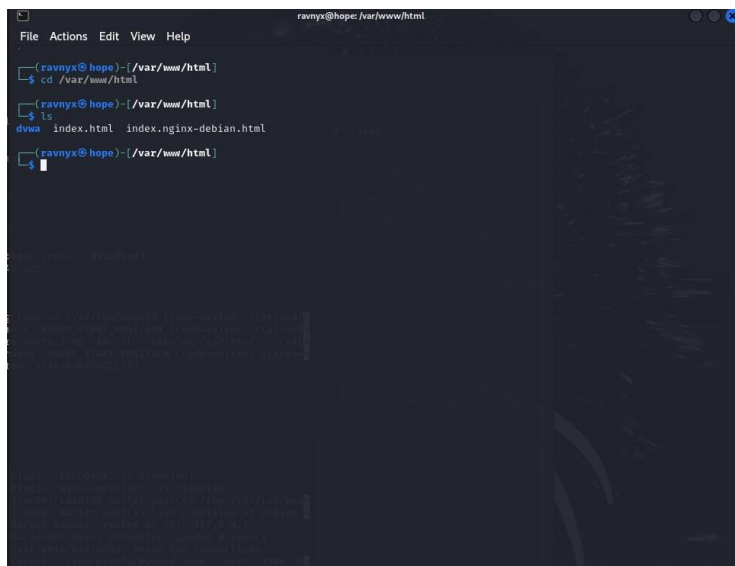
DVWA is an intentionally vulnerable PHP-based web application designed for security training and testing purposes. It contains multiple real-world web vulnerabilities, including SQL Injection and Cross-Site Scripting, categorized by difficulty levels.

The application was connected to a backend database named **dvwa**, which stores user credentials and application data.

The DVWA application was installed in the Apache web root directory and configured to connect with the MariaDB database. Database setup was completed successfully using the DVWA setup interface, confirming proper communication between the application and the database server.

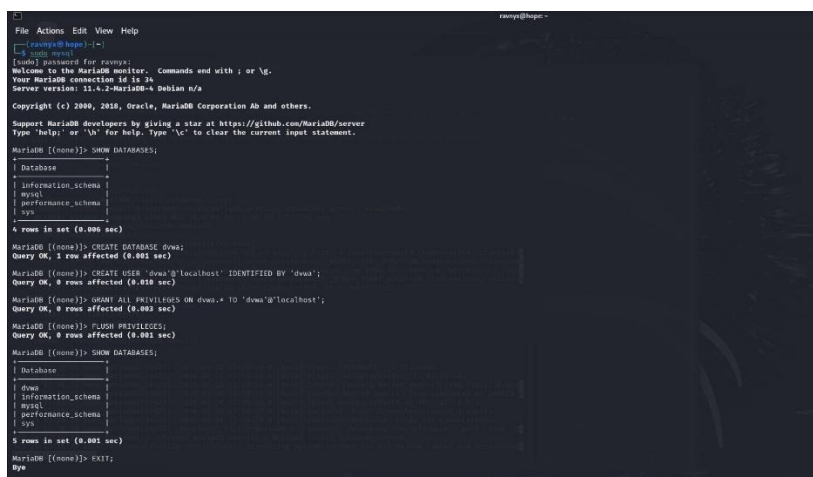
DVWA provides dedicated modules for each vulnerability, enabling structured testing and observation of insecure application behavior.

Figure 3.2: DVWA Application Directory Setup



```
ravnys@hope: /var/www/html
File Actions Edit View Help
[ravnys@hope]~-[ /var/www/html ]
$ cd /var/www/html
[ravnys@hope]~-[ /var/www/html ]
$ ls
dvwa  index.html  index.nginx-debian.html
[ravnys@hope]~-[ /var/www/html ]
$
```

Figure 3.2.1: DVWA Database Setup Successful



```
ravnys@hope: ~
File Actions Edit View Help
[ravnys@hope]~-[ ]
$ sudo mysql
[mysql] password for ravnys:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 11.4.2-MariaDB-0. Debian r/a

Copyright (c) 2009, 2010, Oracle, MariaDB Corporation AB and others.
Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.005 sec)

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';
Query OK, 0 rows affected (0.018 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
Query OK, 0 rows affected (0.000 sec)

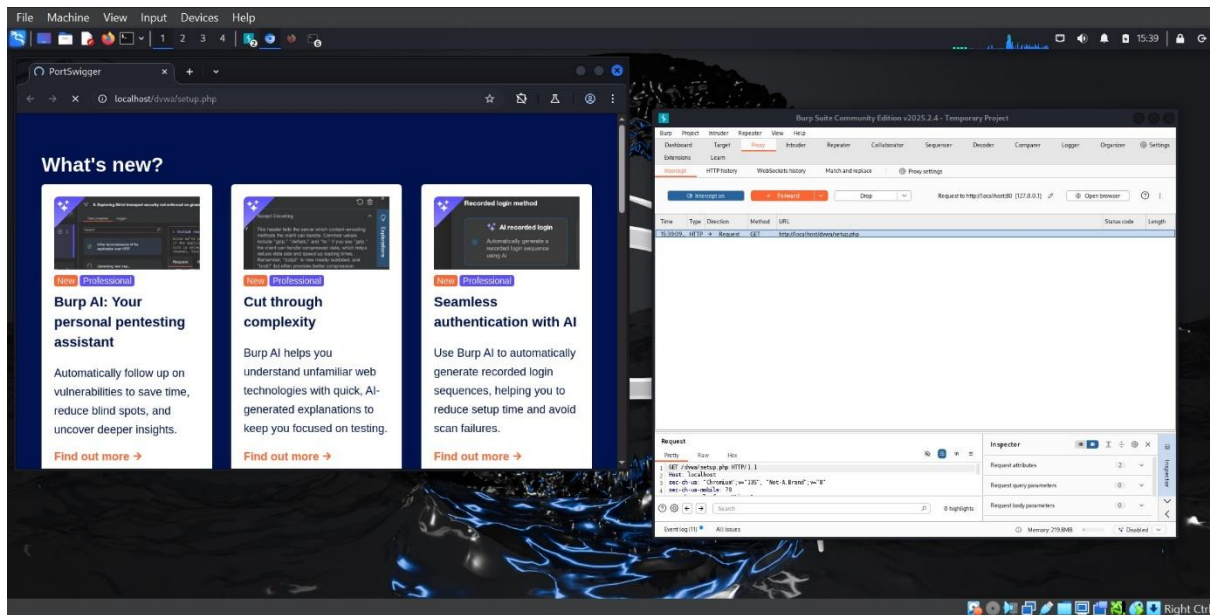
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| dvwa |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
```

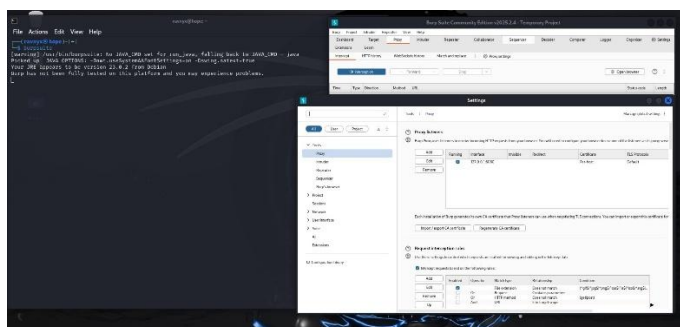
3.3 Burp Suite Configuration

PortSwigger Burp Suite Community Edition was used as the primary interception and analysis tool. Burp Suite acts as an intercepting proxy between the browser and the web application, allowing inspection and modification of HTTP requests and responses.



The Burp Suite embedded browser was used to ensure seamless proxy configuration. Intercept mode was enabled to capture HTTP traffic generated during interaction with DVWA. This setup allowed detailed analysis of request parameters, headers, payloads, and server responses during vulnerability testing.

Figure 3.3: Burp Suite Proxy Configuration



3.4 Proxy and Certificate Setup

To intercept HTTPS traffic, Burp Suite's Certificate Authority (CA) certificate was installed in the browser's trusted certificate store. This ensured encrypted traffic could be decrypted and analyzed without connection errors.

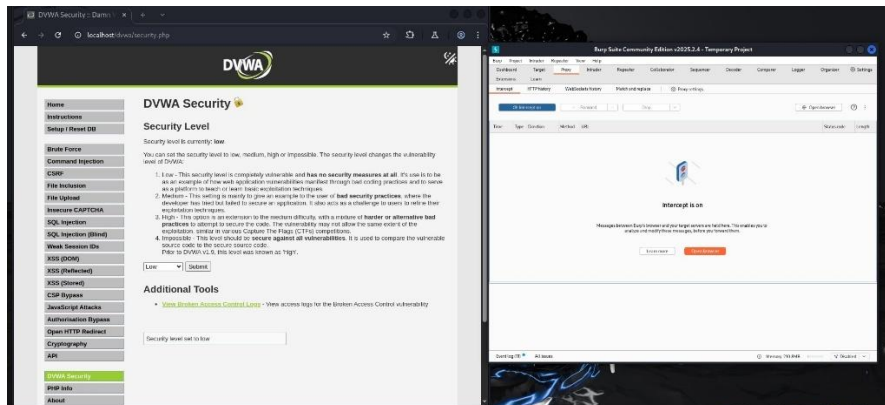
Once configured, all web traffic between the browser and DVWA passed through Burp Suite, enabling effective interception and observation of application behavior during attack simulations.

3.5 DVWA Security Level Configuration

DVWA provides configurable security levels to simulate different degrees of application protection. For the purpose of this assessment, the security level was set to **Low**.

Setting the security level to Low disables most input validation and security controls, allowing vulnerabilities such as SQL Injection and Cross-Site Scripting to be demonstrated clearly. This configuration is commonly used for educational and training purposes.

Figure 3.5: DVWA Security Level Set to Low



Chapter 4. SQL Injection – Analysis and Impact

4.1 Overview of SQL Injection

SQL Injection is a critical web application vulnerability that occurs when an application improperly handles user-supplied input and directly incorporates it into SQL queries. This allows an attacker to manipulate database queries by injecting malicious SQL code through input fields such as login forms, search boxes, or URL parameters.

When input validation and sanitization are absent or weak, attackers can alter the intended logic of database queries, leading to unauthorized access or data manipulation.

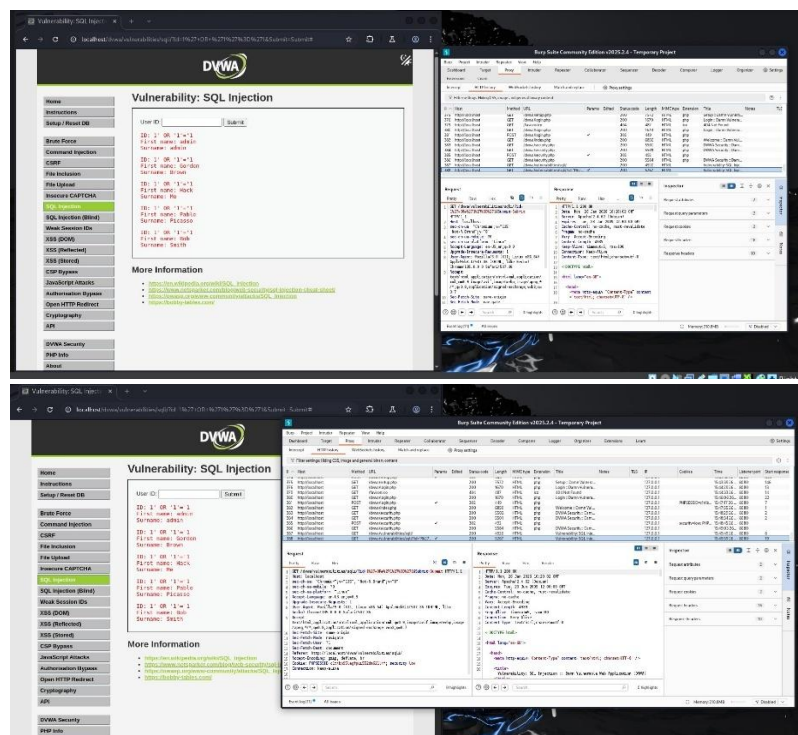
4.2 SQL Injection Testing in DVWA

The SQL Injection module in DVWA was used to demonstrate this vulnerability in a controlled environment. User input was provided through the application's input fields, which were processed directly by backend SQL queries without sufficient validation.

By supplying crafted SQL payloads instead of expected numeric input, the application returned unintended results from the database. This confirmed the presence of an SQL Injection vulnerability. (During testing, a simple SQL injection payload such as `1' OR '1'='1` was used to manipulate the backend SQL query, resulting in unauthorized retrieval of database records.)

The application's behavior clearly demonstrated how user-controlled input influenced backend query execution.

As shown in Figure 4.2: SQL Injection Successful Output



4.3 Observed Behavior and Analysis

During testing, the application responded with database records that should not have been accessible through normal usage. This indicated that the SQL query structure was successfully altered using injected input.

From the intercepted HTTP requests, it was observed that:

- User input was directly included in SQL statements
- No input sanitization or parameterized queries were applied
- The database executed modified queries without validation

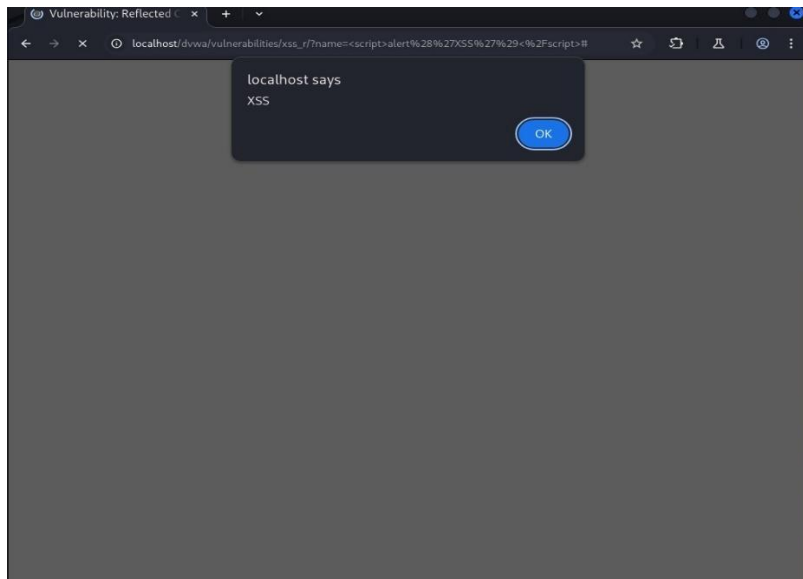
These observations confirm insecure input handling and weak backend query construction.

4.4 Impact of SQL Injection

SQL Injection vulnerabilities pose severe security risks, including:

- Unauthorized access to sensitive data
- Disclosure of user credentials and personal information
- Modification or deletion of database records
- Potential full compromise of the application database
- Escalation to server-level attacks in severe cases

In real-world applications, such vulnerabilities can lead to data breaches, financial loss, and loss of user trust.



5.3 Observed Behavior and Analysis

During testing, the following behaviors were observed:

- User input was reflected directly in the response
- No output encoding was applied
- Injected scripts executed in the browser context

The successful execution of injected JavaScript indicates improper handling of user input on the client side. This confirms the presence of an XSS vulnerability in the application.

5.4 Impact of Cross-Site Scripting

Cross-Site Scripting vulnerabilities can have serious consequences, including:

- Theft of user session cookies
- Account hijacking
- Execution of malicious actions on behalf of users
- Redirection to malicious websites
- Loss of user trust and application credibility

In real-world scenarios, XSS attacks are often used as an entry point for more complex attacks, making them a critical security concern for web applications.

Chapter 6. Observations from Burp Suite Interception

Burp Suite played a crucial role in observing and analysing the communication between the browser and the web application during vulnerability testing. By acting as an intercepting proxy, Burp Suite allowed visibility into HTTP requests and responses generated while interacting with DVWA.

During testing, Burp Suite intercepted both **GET** and **POST** requests generated by the application. GET requests were observed while submitting SQL injection payloads through URL parameters, whereas POST requests were intercepted during authentication and form submissions. This interception allowed detailed inspection of request parameters and server responses.

Key observations from Burp Suite interception include:

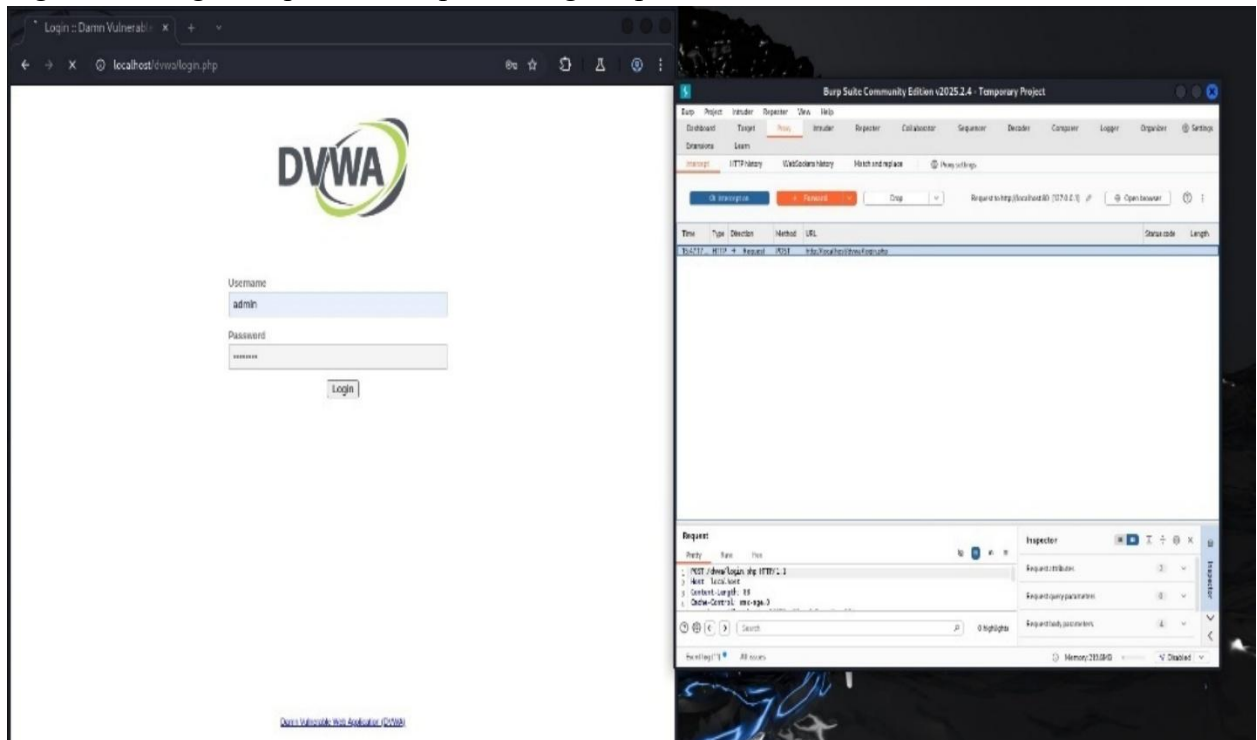
- Request parameters were visible in plain HTTP format, allowing inspection of user input values.
- In the SQL Injection module, the injected input was clearly observable within the intercepted request parameters, confirming that user input was directly passed to backend queries.
- In the XSS module, malicious script input was reflected in the server response, indicating lack of output encoding.
- HTTP responses revealed how the application processed and responded to manipulated input without appropriate security controls.

Burp Suite's HTTP history feature allowed continuous monitoring of application traffic even when interception mode was disabled. This helped in reviewing previously sent requests and correlating application behavior with specific payloads used during testing.

These observations highlight the effectiveness of intercepting proxies in identifying insecure input handling and understanding application logic. The ability to analyse raw HTTP traffic is essential in detecting vulnerabilities that may not be visible through the browser alone.

The following observations were made by correlating browser output with intercepted HTTP requests captured using Burp Suite.

Figure 6.1: Login Request Intercepted Using Burp Suite



Chapter 7. Mitigation and Prevention Techniques

The vulnerabilities identified during testing highlight the importance of implementing secure coding practices and proper security controls in web applications. Effective mitigation techniques can significantly reduce the risk of exploitation and improve overall application security.

7.1 Mitigation of SQL Injection

To prevent SQL Injection attacks, the following measures should be implemented:

- Use **parameterized queries (prepared statements)** instead of dynamically constructing SQL queries.
- Validate and sanitize all user inputs on the server side.
- Apply strict input validation rules for numeric and text-based fields.
- Restrict database user privileges to limit access only to necessary operations.
- Avoid displaying detailed database error messages to users.

These practices ensure that user input is treated as data rather than executable SQL code.

7.2 Mitigation of Cross-Site Scripting (XSS)

To prevent Cross-Site Scripting vulnerabilities, the following controls should be applied:

- Perform proper **output encoding** before displaying user input in web pages.
- Validate and sanitize user inputs to block malicious scripts.
- Use **Content Security Policy (CSP)** headers to restrict execution of untrusted scripts.
- Avoid directly embedding user input into HTML or JavaScript contexts.
- Implement secure frameworks that automatically handle input encoding.

These techniques help prevent the execution of malicious scripts in the user's browser.

7.3 General Web Application Security Practices

In addition to vulnerability-specific controls, general security best practices should be followed:

- Regularly update software components and dependencies.
- Conduct periodic vulnerability assessments and security testing.
- Implement secure authentication and session management mechanisms.
- Follow secure coding standards and guidelines.
- Monitor and log application activity to detect suspicious behavior.

Applying these preventive measures strengthens the overall security posture of web applications and reduces exposure to common attacks.

8. Conclusion

This task successfully demonstrated the process of web application vulnerability testing using a controlled and intentionally vulnerable environment. Through practical experimentation, common web application security flaws such as **SQL Injection** and **Cross-Site Scripting (XSS)** were identified and analyzed.

The use of DVWA provided a safe platform to understand how improper input validation and insecure coding practices can lead to serious security vulnerabilities. Burp Suite proved to be an effective tool for intercepting and analyzing HTTP requests and responses, allowing deeper insight into application behavior and data flow.

The observations from this task emphasize the importance of secure development practices, proper input handling, and regular security testing. Identifying vulnerabilities at an early stage helps prevent exploitation and protects sensitive data from unauthorized access.

Overall, this task enhanced practical understanding of web application security concepts and highlighted the need for proactive vulnerability assessment to build secure and reliable web applications.